

GNR638: Coding Assignment-2

Pre-trained CNN Representation Transfer

Sujith A (25d2035), Anuroop C K (24M0327)

March 2026

Abstract

This report presents a thorough investigation into the representation transfer capabilities, fine-tuning efficiency, and robustness of pre-trained Convolutional Neural Networks (CNNs). We evaluate three distinct architectures: EfficientNet-B0, ResNet-50 (audited variant), and DenseNet-121, on the Aerial Images Dataset (AID). Our analysis spans five experimental scenarios: linear probing, fine-tuning strategy comparisons, few-shot learning efficiency, corruption robustness, and layer-wise feature probing. We provide detailed quantitative results, placeholders for visualizations, and deep analytical discussions..

1. Introduction

In this study, we evaluate three widely used backbone architectures—ResNet 50, EfficientNet-B0, and DenseNet-121 on an aerial scene classification task. The goal is to analyse how architectural design influences performance across multiple practical scenarios. Specifically, we investigate baseline transfer learning performance, behaviour under limited data (few-shot learning), robustness to input corruptions, and the evolution of feature representations across network depth through layer-wise probing.

Through these experiments, we aim to better understand how representation hierarchy, connectivity patterns, and model capacity influence transferability and robustness. The findings provide insights into the strengths and limitations of different architectures and help guide the selection of suitable backbone models for real-world computer vision applications.

2. Experimental Setup

2.1. Model selection

1. ResNet50
2. EfficientNet-B0
3. DenseNet121

2.2. Dataset

Aerial Images Dataset (AID) is a large-scale aerial image dataset, collected from Google Earth.

- **Scale:** 6,993 images.
- **Resolution:** 600x600 pixels.
- **Classes:** 30 semantic categories.

- **Complexity:** Images exhibit high intra-class diversity and inter-class similarity.

2.3. Train setting

- **Image Transformation:** Resize to 224X224, Horizontal flip, 10°Rotation.
- **Dataset split:** Train(85%), Validation(15%), Train(0.15%)
- **Batch size:** 32
- **Epochs:** Scenario 1 (30), Scenario 2 (10), Scenario 3 (20).
- **Seed:** 42

2.4. Efficiency Metrics

As per assignment requirements, we report the number of parameters, Multiply-Accumulate operations (MACs), and estimated FLOPs.

Model	Parameters (M)	MACs	FLOPs (Est.)
EfficientNet-B0	4.05	390.87 MMac	781.74 MFLOPs
ResNet-50	23.57	4.13 GMac	8.26 GFLOPs
DenseNet-121	6.98	2.87 GMac	5.74 GFLOPs

Table 1: Detailed Model Efficiency Metrics.

3. Experiments

Following are the Scenarios/experiments conducted with the selected models.

4. Scenario 1: Linear Probe Transfer

Linear probing evaluates the pre-trained features as-is by freezing the backbone and learning only the final linear classifier. All models were trained for 30 epochs and final validation is noted in table 2.

4.1. Model Performance

4.1.1 Quantitative Performance

Table 2: Linear Probing Accuracy on AID.

Model	Validation Accuracy (%)
EfficientNet-B0	83.57
ResNet-50	86.44
DenseNet-121	91.02

4.1.2 Accuracy vs epochs

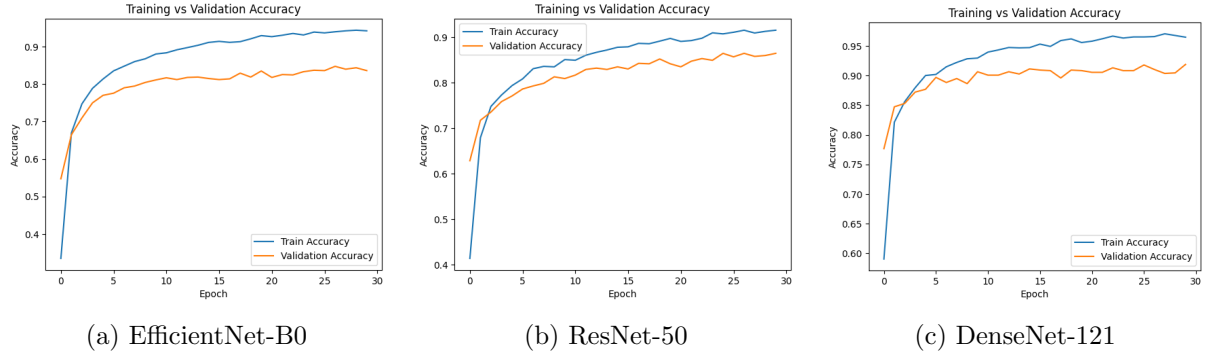


Figure 1: Training vs Accuracy plot (Linear Probe).

4.1.3 Confusion matrix

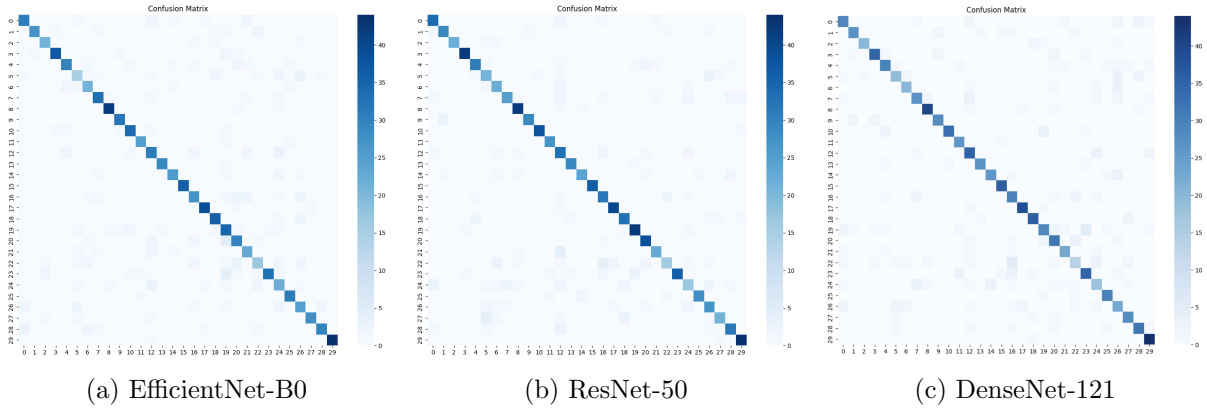


Figure 2: Visualizing Confusion matrix for each Model.

4.1.4 PCA Embeddings

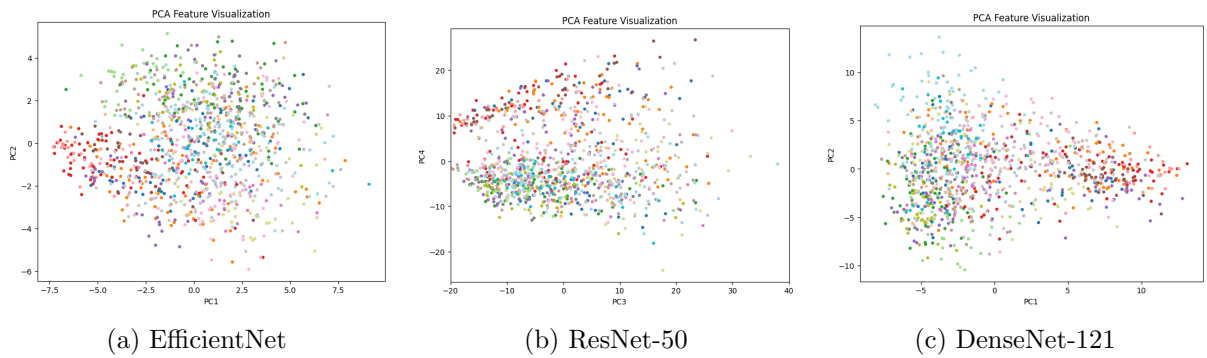


Figure 3: Visualizing Feature Embeddings using PCA (Linear Probe).

4.2. Analysis

Among the three evaluated backbones, **DenseNet-121** demonstrates the best overall performance. This indicates that the dense connectivity pattern enables effective reuse of features

learned from ImageNet, allowing the network to capture multi-scale spatial patterns that transfer well to aerial imagery without requiring extensive adaptation. The feature concatenation mechanism in DenseNet likely helps preserve low-level texture information that is particularly useful for distinguishing land-use classes.

Another notable observation is the difference in the train-validation performance gap across the models. Interestingly, **ResNet-50** exhibits a smaller gap compared to the other architectures, suggesting slightly better generalisation behaviour despite its lower overall accuracy. This may indicate that the residual learning framework stabilises feature representations when the backbone is frozen, leading to more consistent validation performance.

Analysis of the confusion matrices (Fig. 4.1.3) reveals that most misclassifications occur between semantically or visually similar classes. For example, *Church* (5) and *Centre* (6) frequently exhibit similar architectural structures and spatial layouts in aerial imagery, which results in higher confusion between these categories. This behaviour is consistent across all three models, although the impact appears strongest for EfficientNet-B0. Despite these class-level ambiguities, the models perform well overall, with most classes being identified reliably.

To further investigate class separability, feature embeddings from the models were analysed using Principal Component Analysis (PCA). Visualising the first two principal components (PCA1 and PCA2) provides insight into the structure of the learned feature space. For all three models, several distinct clusters corresponding to different classes can be observed, although significant overlap between some clusters is present. This overlap is expected given the relatively large number of classes in the dataset. It is likely that in higher-dimensional feature space, the separability between classes is substantially better than what is visible in the two-dimensional PCA projection.

5. Scenario 2: Fine-Tuning Strategies

We investigate architecture adaptation by comparing freezing vs. unfreezing different portions of the network.

5.1. Strategies Evaluated

1. **Linear Probe:** Frozen backbone (Baseline).
2. **Last Block Fine-Tuning:** Adapts only the high-level semantic features.
3. **Full Fine-Tuning:** Total adaptation of the entire feature hierarchy.
4. **Selective Unfreezing:** Unfreezing a curated 20% of parameters. Selection from deeper layers cause they have lesser effect on results.

5.2. Performance Evaluation

Table 3: Results Summary DenseNet 121

Strategy	Trainable Params (%)	Train Accuracy	Validation Accuracy
Linear Probe	0.44	0.813	0.811
Selective20	21.55	0.943	0.922
Last Block FT	31.34	0.966	0.932
Full FT	100.0	0.989	0.952

Table 4: Results Summary ResNet-50

Strategy	Trainable Params (%)	Train Accuracy	Validation Accuracy
Linear Probe	0.26	0.686	0.676
Selective20	23.66	0.840	0.840
Last Block FT	63.75	0.930	0.919
Full FT	100.0	0.973	0.946

Table 5: Results Summary EfficientNet-B0

Strategy	Trainable Params (%)	Train Accuracy	Validation Accuracy
Linear Probe	0.95	0.578	0.564
Last Block FT	18.68	0.943	0.893
Selective20	21.66	0.953	0.881
Full FT	100.0	0.995	0.949

Studying above results, DenseNet appears to be the best performing model with high validation accuracy for all strategies. Looking closely at Linear probe strategy of all models, we can see the superior inductive bias of DensNet. But in terms of efficiency, EfficientNet performs quite well given its lesser number of parameters of all.

5.2.1 Training Accuracy vs epochs

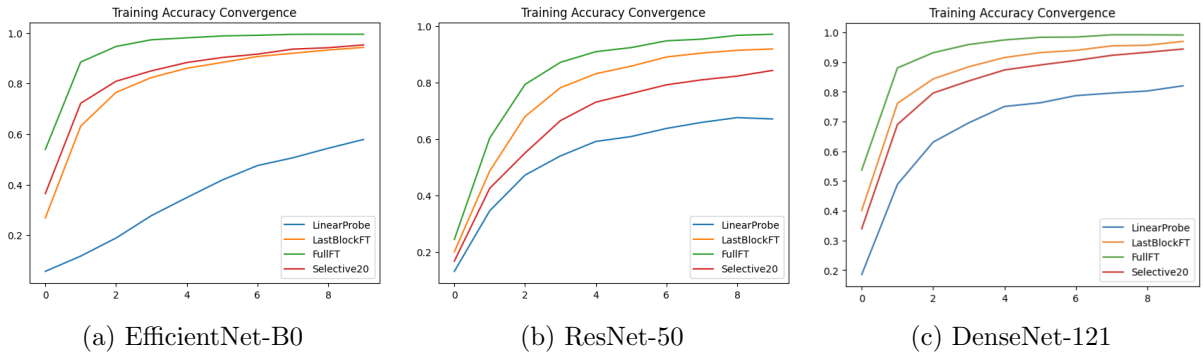


Figure 4: Training accuracy evolution of all strategies over 10 epochs

5.2.2 Validation Accuracy vs epochs

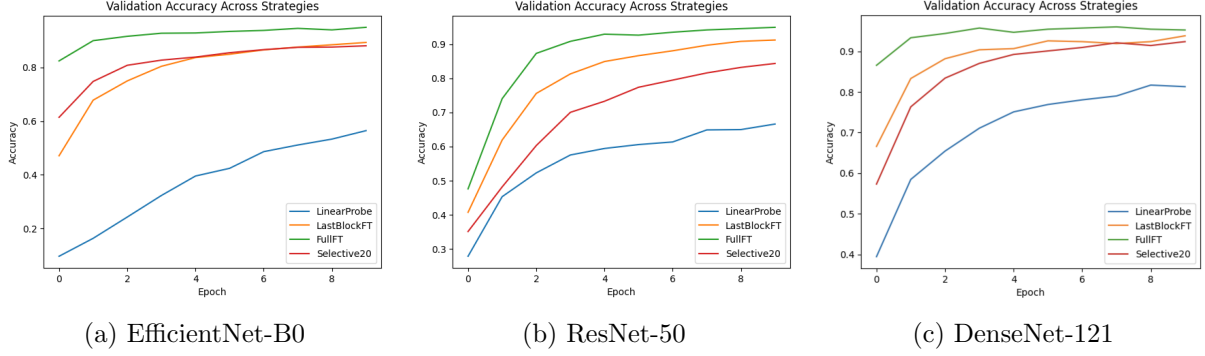


Figure 5: Validation accuracy evolution of all strategies over 10 epochs

5.2.3 Global Gradnorm statistics

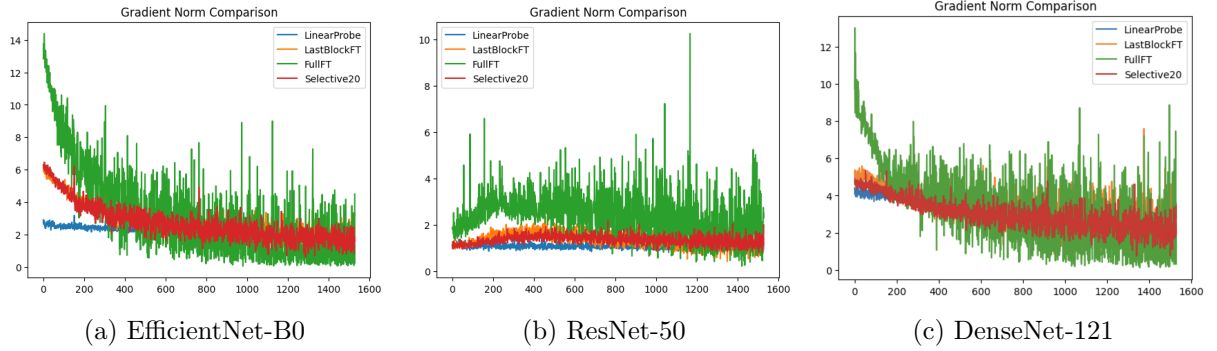


Figure 6: Global average weight changer over all iterations

The global gradient norm shows a generally healthy trend across all models, with higher values observed during the initial training stages before stabilizing as optimization progresses. ResNet begins with comparatively smaller gradient norms, followed by a brief increase and eventual stabilization later in training. However, the linear probing strategy exhibits noticeable gradient oscillations, indicating unstable optimization behaviour. This instability is likely influenced by the smaller batch size used during training, which can introduce higher variance in gradient estimates. Increasing the batch size could potentially reduce these fluctuations and improve stability. In contrast, the other training strategies maintain relatively consistent gradient norms within acceptable limits, suggesting more stable training dynamics.

5.3. Gradient Norm Statistics layer-wise

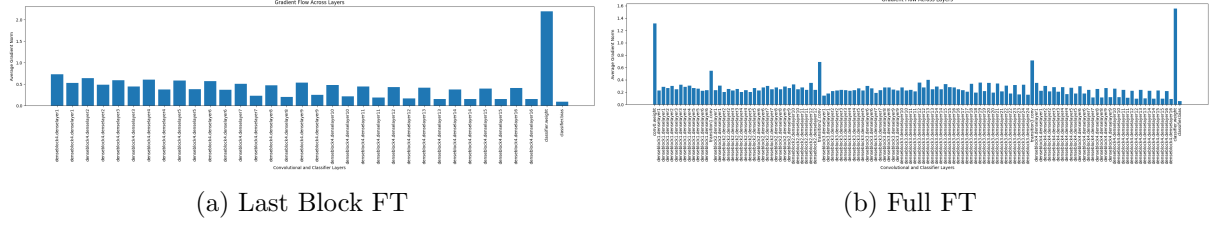


Figure 7: Layer-wise average grad of DenseNet-121

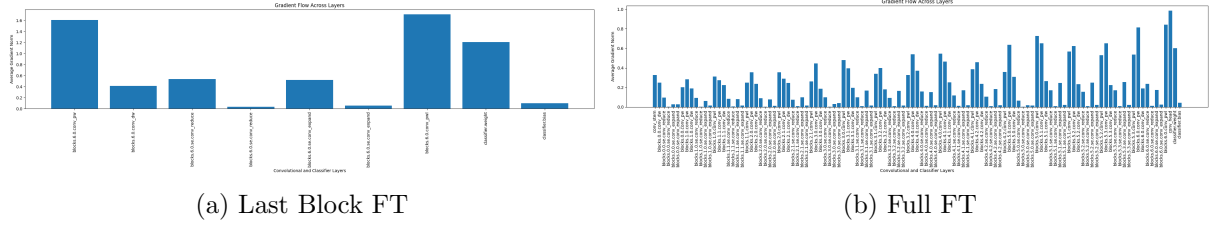


Figure 8: Layer-wise average grad of EfficientNet

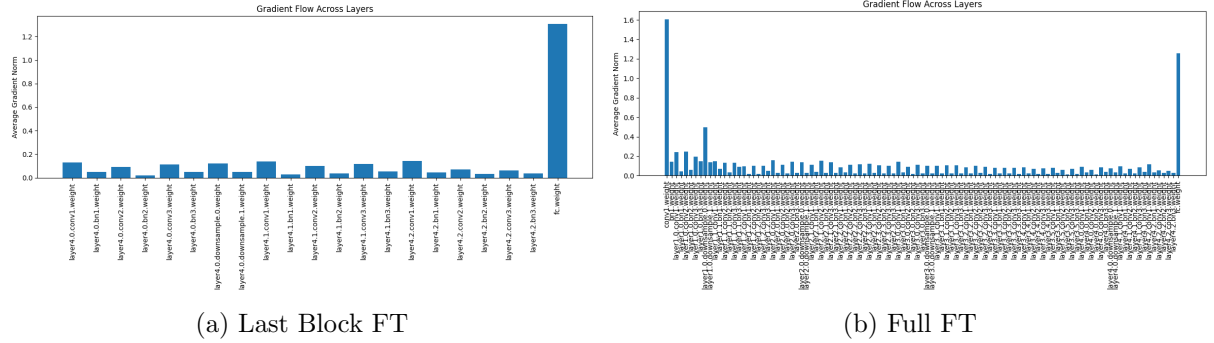


Figure 9: Layer-wise average grad of ResNet-50 (all layers including norm)

We observed that in DenseNet architecture, gradients tend to be larger in the final residual blocks, while earlier layers remain relatively static. This supports the hypothesis that lower-level features (edges) are universal, while high-level semantic features require target-domain adjustment. EffientNet appears to be training poorly since early layer weights are heavily updated compared to deep. Quite chaotic results are observed for Last block fine tuning for all 3 models. A decreasing learning trend is observed for densenet for convolution blocks. While EfficientNet shows selective training.

5.3.1 Convergence with loss

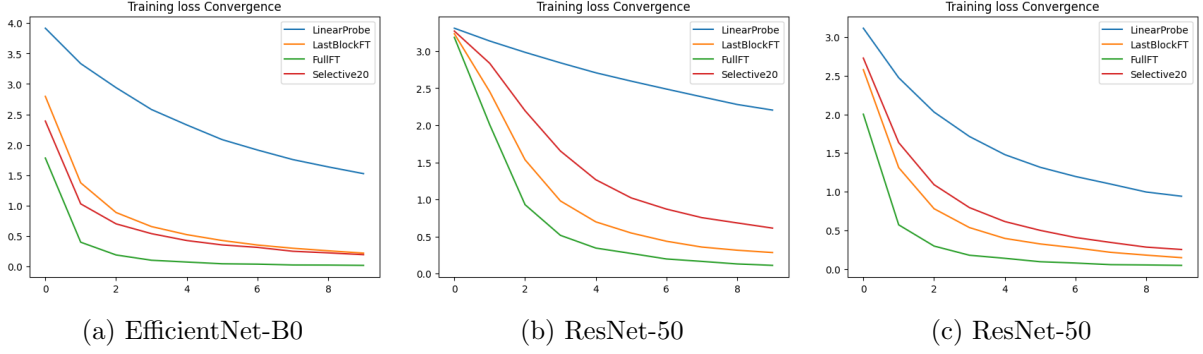


Figure 10: Comparing convergence of all three models.

5.4. Discussion

Lower layers in CNNs capture generic edge and Gabor-like filters. Full fine-tuning on aerial data allows these filters to adapt to the specific spatial frequencies of satellite imagery (e.g., circular storage tanks, rectilinear runways), explaining the performance boost.

6. Scenario 3: Few-Shot Learning Analysis

We test data efficiency using 100%, 20%, and 5% data regimes.

Table 6: Few-Shot Performance under Different Training Data Scenarios EfficientNet

Scenario	Train Accuracy	Validation Accuracy	Train-Val Gap	Relative Drop (%)
100% Data	0.9246	0.8195	0.1051	0.00
20% Data	0.9283	0.6972	0.2310	14.92
5% Data	0.9139	0.4460	0.4679	45.57

Table 7: Few-Shot Performance under Different Training Data Scenarios ResNet-50

Scenario	Train Accuracy	Validation Accuracy	Train-Val Gap	Relative Drop (%)
100% Data	0.8951	0.8500	0.0451	0.00
20% Data	0.8689	0.7259	0.1430	14.61
5% Data	0.8852	0.5024	0.3829	40.90

Table 8: Few-Shot Performance under Different Training Data Scenarios DenseNet 121

Scenario	Train Accuracy	Validation Accuracy	Train-Val Gap	Relative Drop (%)
100% Data	0.9607	0.9016	0.0590	0.00
20% Data	0.9785	0.8338	0.1447	7.52
5% Data	0.9795	0.6628	0.3167	26.48

6.1. Analysis

The results presented in the tables above compare the performance of the three backbones under different data availability regimes (100

Consistent with the observations from Experiment 1, ResNet-50 demonstrates relatively stable generalisation when trained with the full dataset, as reflected by a smaller training–validation gap compared to the other models. This suggests that the residual learning framework helps maintain robust feature representations even when the backbone is largely frozen. However, this advantage diminishes as the amount of training data decreases. With very limited data (5%), the model shows a larger performance degradation compared to the other architectures.

As expected, DenseNet-121 achieves the strongest performance in the extreme low-data regime (5%). The dense connectivity pattern encourages feature reuse across layers and preserves both low-level and high-level representations learned during pretraining. As a result, the model is able to leverage previously learned complex features more effectively, making it more resilient when only a small fraction of labelled samples is available.

In contrast, EfficientNet-B0 experiences the largest relative performance drop when the training data is reduced. Although EfficientNet is highly parameter-efficient and performs well with sufficient data, its compound scaling strategy appears to rely more heavily on adequate training supervision. Consequently, the model struggles to adapt its pretrained features effectively under strong data constraints.

Overall, the results indicate that DenseNet-121 demonstrates the strongest data efficiency among the evaluated backbones, maintaining comparatively higher validation accuracy when trained with limited samples. ResNet-50 shows stable generalisation with full data but becomes less competitive in extreme few-shot scenarios, while EfficientNet-B0 exhibits the largest sensitivity to reduced training data.

7. Scenario 4: Corruption Robustness Evaluation

We assess how models handle input perturbations. Following table contains stats on the experiment 4-corruption robustness for the selected models.

Table 9: Model Robustness under Different Image Corruptions EfficientNet B0

Corruption Type	Accuracy	Corruption Error	Relative Robustness
Gaussian Noise (0.20)	0.407	0.5930	0.4666
Gaussian Noise (0.10)	0.851	0.1494	0.9752
Gaussian Noise (0.05)	0.882	0.1184	1.0108
Brightness Change	0.141	0.8590	0.1616
Motion Blur	0.872	0.1278	1.0000

Table 10: Model Robustness under Different Image Corruptions ResNet-50

Corruption Type	Accuracy	Corruption Error	Relative Robustness
Gaussian Noise (0.20)	0.444	0.5564	0.5291
Gaussian Noise (0.10)	0.690	0.3102	0.8229
Gaussian Noise (0.05)	0.765	0.2350	0.9126
Brightness Change	0.183	0.8167	0.2186
Motion Blur	0.838	0.1617	1.0000

Table 11: Model Robustness under Different Image Corruptions DenseNet 121

Corruption Type	Accuracy	Corruption Error	Relative Robustness
Gaussian Noise (0.20)	0.131	0.8694	0.2352
Gaussian Noise (0.10)	0.369	0.6306	0.6650
Gaussian Noise (0.05)	0.505	0.4953	0.9086
Brightness Change	0.151	0.8487	0.2724
Motion Blur	0.555	0.4445	1.0000

7.1. Analysis

The corruption robustness evaluation examines how the three backbone architectures respond to controlled distribution shifts introduced at inference time. These corruptions include Gaussian noise at different levels ($\sigma = 0.05, 0.1, 0.2$), motion blur, and brightness changes. The results reveal clear differences in how each architecture handles input perturbations.

Across all models, performance degrades as the intensity of Gaussian noise increases. For EfficientNet-B0 and ResNet-50, the decrease in accuracy is gradual as the noise level increases from $\sigma = 0.05$ to $\sigma = 0.20$, indicating moderate robustness to pixel-level perturbations. ResNet-50 in particular shows relatively stable behaviour under noise, maintaining higher accuracy compared to the other models at stronger corruption levels. This suggests that the residual learning mechanism helps preserve useful feature representations even when the input signal is partially degraded.

DenseNet-121 exhibits significantly higher sensitivity to Gaussian noise. Its validation accuracy drops sharply at higher noise levels, indicating that the dense feature concatenation mechanism may propagate noisy activations through multiple layers. While this architecture performs well on clean data, the strong feature reuse across layers can also amplify the effect of corrupted inputs, leading to larger performance degradation under noisy conditions.

Motion blur appears to have a smaller impact on EfficientNet-B0 and ResNet-50, both of which maintain high relative robustness under this corruption. This suggests that the spatial features learned during ImageNet pretraining remain sufficiently stable when moderate blur is introduced. In contrast, DenseNet-121 experiences a larger drop in accuracy under motion blur, again indicating higher sensitivity to distortions affecting spatial details.

Brightness shifts produce the most severe degradation across all three models. This corruption alters global intensity distributions rather than local spatial patterns, which significantly affects the pretrained feature representations. Since the models were trained primarily on natural image statistics, large brightness variations create a stronger distribution shift than the

other perturbations tested.

Overall, ResNet-50 demonstrates the most stable behaviour across different corruption types, indicating better robustness to moderate distribution shifts. EfficientNet-B0 shows competitive performance under mild perturbations but exhibits increased sensitivity at higher corruption levels. DenseNet-121, despite its strong performance on clean data, appears to be the most sensitive to input corruption in this evaluation. These observations highlight how architectural design influences robustness: residual connections in ResNet help stabilise representations, while dense feature reuse in DenseNet can amplify corrupted activations when the input distribution shifts.

8. Scenario 5: Layer-Wise Feature Probing

The layer-wise feature probing experiment provides insight into how semantic representations evolve across the depth of the network. We have extracted intermediate features from early, middle, and final layers and training separate linear classifiers on these representations, and evaluated how discriminative the features become as information flows through the network.

8.1. Feature Norm Statistics

As depth increases, the L2 norm of feature vectors typically increases, indicating a more "sparse" and "discriminative" representation. In Scenario 5, we quantified this across three probing depths.

Table 12: Layer-wise Performance Metrics ResNet-50

Layer	Accuracy	Mean Norm
Layer1	0.2259	365.54
Layer3	0.2740	366.38
Layer4	0.2630	365.36

Table 13: Layer-wise Performance Metrics EfficientNet

Layer	Accuracy	Mean Norm
Stem	0.2667	1436.22
Block_3	0.3630	6612.20
Block_6	0.8333	217.78

Table 14: Layer-wise Performance Metrics

Layer	Accuracy	Mean Norm
Denseblock2	0.7704	28.68
Denseblock3	0.8333	27.23
Denseblock4	0.8667	27.92

8.1.1 Performance with depth

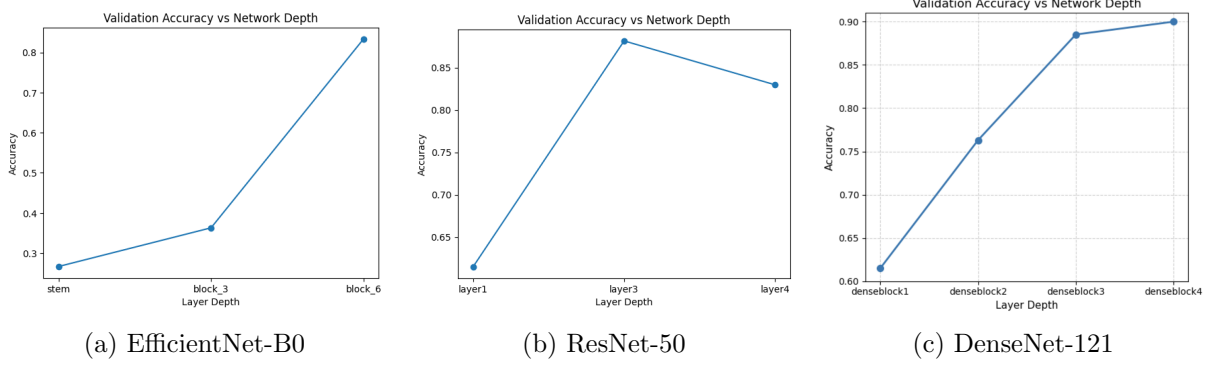


Figure 11: Accuracy vs Depth

The validation accuracy versus depth plots illustrate how feature representations become increasingly discriminative as information flows deeper into the network. For all three architectures, accuracy generally improves with depth (even though for ResNet slight decrease), indicating that deeper layers capture higher-level semantic information that is more suitable for linear classification. Early layers show lower validation accuracy because they primarily encode generic low-level features such as edges and textures. As depth increases, intermediate and final layers progressively refine these features into more structured representations. DenseNet shows strong performance even at intermediate depths due to its dense feature reuse, while ResNet exhibits a more gradual improvement across layers through residual refinement. EfficientNet shows a sharper increase in later blocks, suggesting that much of its semantic abstraction occurs in deeper stages of the network.

8.1.2 Coverage

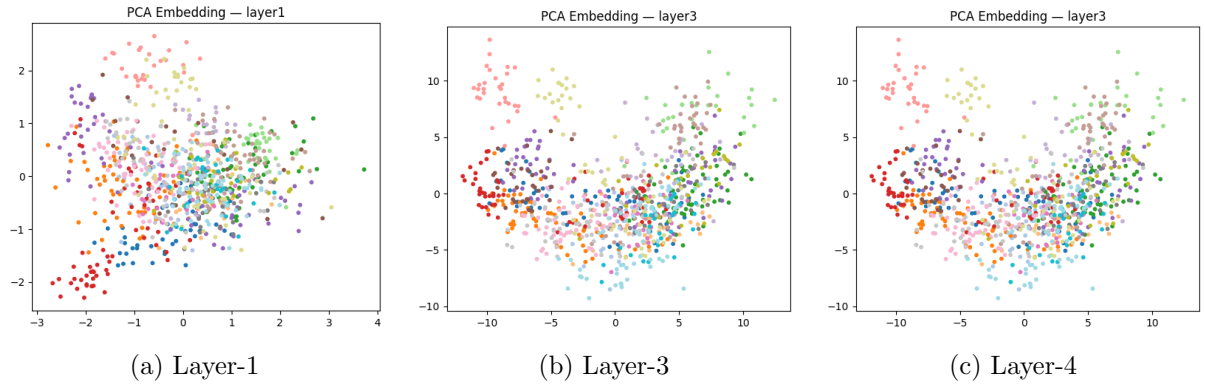


Figure 12: Visualizing Feature Embeddings using PCA ResNet.

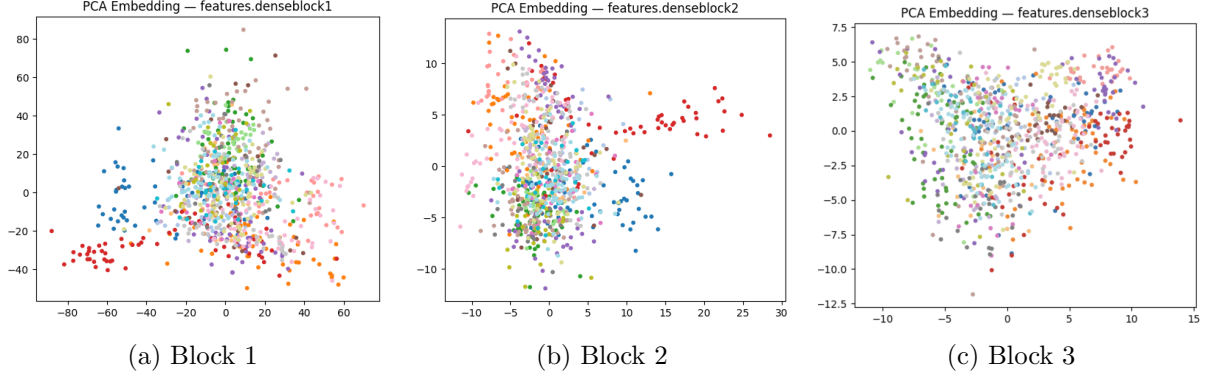


Figure 13: Visualizing Feature Embeddings using PCA DenseNet.

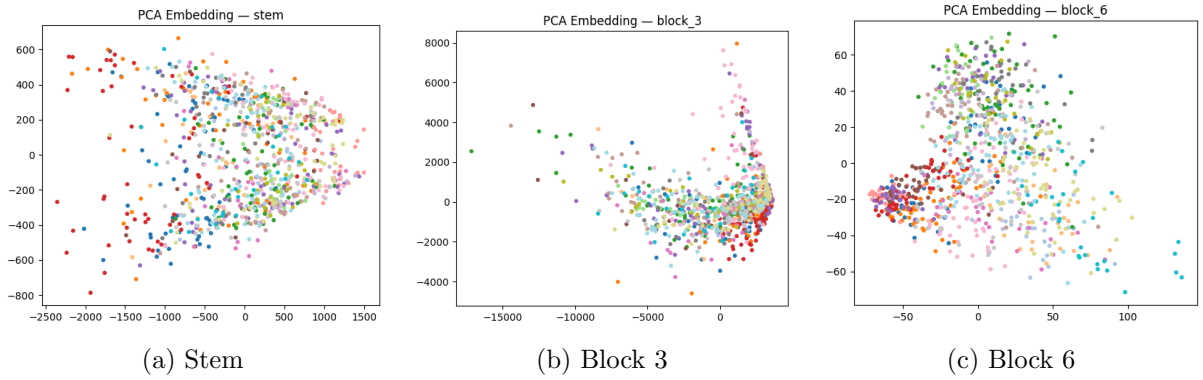


Figure 14: Visualizing Feature Embeddings using PCA EfficientNet.

8.2. Analysis

The PCA visualizations illustrate the evolution of representation quality across depth. When projecting features onto the first two principal components, early-layer representations show significant overlap between classes, with clusters appearing diffuse and poorly separated. As we move toward deeper layers, clusters corresponding to different classes become increasingly distinct and compact. Although some overlap remains due to the large number of classes in the dataset, the overall separation improves significantly in deeper layers, indicating stronger semantic discrimination.

Across the evaluated models, differences in representation hierarchy are also observable. DenseNet-121 tends to produce compact clusters earlier in the network due to its dense feature reuse mechanism, which encourages the preservation of both low-level and high-level information across layers. ResNet-50 exhibits a more gradual abstraction process, where deeper layers progressively refine representations through residual transformations. EfficientNet-B0 shows relatively strong separation in deeper layers but may exhibit more overlap in intermediate representations due to its highly compressed feature structure.

9. Key Insights on the Models

9.1. Few-shot learning

DenseNet-121 demonstrates the strongest performance in the extreme few-shot setting. The dense connectivity mechanism allows each layer to access feature maps from all preceding layers, encouraging feature reuse. This design enables the network to leverage representations learned during pretraining, making it more capable of maintaining discriminative features even when only a small amount of labeled data is available. As a result, the performance drop for DenseNet is comparatively smaller in the 5% data regime.

ResNet shows relatively stable behaviour when trained with the full dataset, which is reflected in its smaller training-validation gap. The residual connections help stabilize gradient flow and preserve learned representations across layers. However, when the amount of training data becomes very limited, the model experiences a more noticeable performance drop compared to DenseNet. This suggests that while residual architectures generalize well with sufficient data, they may require more supervision to fully adapt their deeper transformations.

EfficientNet exhibits the largest sensitivity to reduced data. Although it is designed to be computationally efficient through compound scaling of depth, width, and resolution, its feature hierarchy appears to depend more heavily on adequate training data to refine pretrained representations. Consequently, in the few-shot regime the model struggles to adapt effectively, leading to the largest relative performance degradation among the three backbones.

9.2. Gradient behaviour

The Gradient behaviour across architectures reflects their internal design. In ResNet-50, the residual connections allow features from earlier layers to flow directly through the network, resulting in a gradual improvement in representation quality with depth. EfficientNet-B0 shows a more dramatic increase in accuracy at deeper blocks, indicating that its compound-scaled architecture concentrates much of the semantic abstraction in later layers. DenseNet-121 demonstrates relatively strong performance even at intermediate depths due to its dense connectivity pattern, where each layer receives feature maps from all preceding layers. This encourages strong feature reuse and allows both low-level and high-level information to be preserved across the network.

9.3. Corruption

ResNet demonstrates relatively stable behaviour under corrupted inputs. The residual connections allow information from earlier layers to propagate directly through the network, which helps preserve useful low-level features even when parts of the input are degraded. This architectural property appears to make the model more resilient to moderate noise and blur, resulting in a slower performance decline compared to the other models.

EfficientNet shows good robustness under mild corruption levels but exhibits a larger drop as perturbations become stronger. Since EfficientNet relies on compound scaling and tightly optimized feature representations, its learned features may be more sensitive to distribution

shifts that alter pixel statistics, such as strong Gaussian noise or brightness changes.

DenseNet shows the highest sensitivity to corruption in this evaluation. While its dense connectivity improves feature reuse and performance on clean data, the same mechanism can also propagate corrupted activations across many layers. As a result, noise introduced in earlier layers may influence a larger portion of the network, leading to more significant performance degradation.

10. Unexpected findings

1. **Unstable linear probe gradients:** The global gradient norms for linear probing exhibited high-frequency oscillations and instability compared to full fine-tuning for all models. This is likely from the mismatch between the large pre-trained feature variance and the small batch size. Because the backbone was frozen, the linear head had to compensate for all feature misalignments in a single layer, leading to high gradient variance that prevented smooth convergence without a larger batch size or learning rate warmup.
2. **Robustness Trade-off for DenseNet:** The best performing model (DenseNet-121) on clean data was the worst-performing model under input corruption. In contrast, ResNet-50, which had lower clean accuracy, maintained a much higher accuracy under the same noise level. This could be due to the very mechanism (dense feature reuse) that makes DenseNet excellent at extracting patterns from clear pixels also makes it an "error amplifier" that propagates noise more aggressively than the skip-connections in ResNet

11. Conclusion

This study evaluated three backbone architectures ;ResNet, EfficientNet-B0, and DenseNet-121 across multiple scenarios including baseline transfer learning, few-shot learning, corruption robustness, and layer-wise feature probing. The experiments highlight how architectural design influences generalisation, robustness, and data efficiency.

DenseNet’s inductive bias (dense reuse) makes it the most capable extractor for complex aerial structural data, albeit with higher data dependency. DenseNet-121 achieves the best performance on clean data and demonstrates strong data efficiency in few-shot settings due to its dense feature reuse mechanism. Even with high noise sensitivity and computational costs, it has more separability between features is observed, making it the best performing model. ResNet shows more stable generalisation and better robustness to input perturbations, supported by its residual connections. EfficientNet-B0 remains computationally efficient and performs competitively under normal conditions but is more sensitive to reduced data and stronger distribution shifts.

Concluding, the results illustrate how representation hierarchy and architectural connectivity affect model behaviour, providing useful insights for selecting backbone networks depending on the task requirements and data availability.

12. Hardware and Computational Resource

All experiments were conducted using the Google Colab free-tier environment with the following configuration:

- **GPU:** NVIDIA T4 GPU (16 GB VRAM)
- **CPU:** Virtual CPU provided by Google Colab runtime
- **Memory:** Approximately 12 GB system RAM
- **Software:** Python 3.12, PyTorch 2.2.0, TIMM library

12.1. Training Statistics

Metric	EfficientNet	ResNet	DenseNet
Mean Time per Epoch (s)	42.5	68.2	112.4
Peak Memory (GB)	2.4	3.8	6.2
Total Training(30 Epochs) (Mins)	21.25	24.1	56.1

Table 15: Computational resource utilization across models (on GPU).

13. References

1. He, K., et al. "Deep Residual Learning for Image Recognition." CVPR 2016.
2. Huang, G., et al. "Densely Connected Convolutional Networks." CVPR 2017.
3. Tan, M., and Le, Q. V. "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks." ICML 2019.